VARDHAMAN
COLLEGE OF ENGINEERING

A Course End Project Report on

Efficient Route Optimization Using Data Structures and Algorithms

A9504- DATA STRUCTURES

Submitted by

P. Shivaji Goud
(25881A0554)

Course Facilitator

Dr. V. Lokeshwari Vinya
Associate Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering,

Hyderabad

April 2026

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,**
Approved by **AICTE,** Accredited by **NAAC** with **A**++ Grade
Kacharam, Shamshabad, Hyderabad- 501 218,
Telangana, India.

Vibrant Vardhaman

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Efficient Route Optimization Using Data Structures and Algorithms"** is carried out by **"P. Shivaji Goud" with** Roll Numbers **"25881A0554"** towards **A9504 – Data Structures Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2025-26.

Signature of the Course Faculty
Dr. V. Lokeshwari Vinya
Assoc. Prof, CSE

Signature of the HOD
Dr. Ramesh Karnati
Assoc.Prof. HOD. CSE

Contents

Abstract

The Route Planner / Map Navigator project is implemented using core C++ and data structures, where the road network is modelled as a graph. Each city is represented as a node, and the roads connecting them are treated as weighted edges based on distance. An adjacency list is used to efficiently store the graph, and a priority queue (min-heap) is applied to optimize pathfinding. The system uses Dijkstra's Algorithm to compute the shortest path between a selected source and destination. During execution, the algorithm initializes distances, repeatedly selects the nearest unvisited node, and updates neighbouring distances until the shortest route is determined.

The implementation includes features such as adding cities and routes, displaying connections, and computing the shortest path along with total distance. The output also reconstructs the path using a parent array, providing a clear route from source to destination. The project can be enhanced with a menu-driven interface, file handling for persistent storage, and the use of city names instead of numeric indices for better usability. Overall, this project demonstrates practical application of graphs, priority queues, and algorithmic problem-solving in real-world navigation systems.

Keywords: Graph Theory, Dijkstra's Algorithm, Shortest Path Computation, Route Optimization, Adjacency List, Priority Queue, Weighted Graph, Path Reconstruction.

1 Introduction

The implementation of the **Route Planner / Map Navigator** project is grounded in fundamental concepts of graph theory and data structures. The transportation network is modelled as a **weighted graph**, where cities are represented as vertices (nodes) and roads as edges with associated distances (weights). To efficiently store and traverse this graph, an **adjacency list** representation is used, which optimizes memory usage compared to an adjacency matrix, especially for sparse graphs. The project relies heavily on Dijkstra's Algorithm, a greedy algorithm used to compute the shortest path from a source node to all other nodes in a graph with non-negative edge weights. The algorithm works by iteratively selecting the node with the minimum tentative distance and updating the distances of its neighbouring nodes, a process known as relaxation. A **priority queue (min-heap)** is used to efficiently extract the node with the smallest distance, improving the time complexity to **O(E log V)**, where *E* is the number of edges and *V* is the number of vertices.

From a mathematical perspective, the project involves concepts such as graph traversal, optimization, and distance minimization. The correctness of the algorithm is based on the principle that once the shortest distance to a node is finalized, it cannot be improved further under non-negative weight conditions. The project also uses arrays or vectors to maintain distance values and parent relationships, enabling **path reconstruction** from the destination back to the source. Basic discrete mathematics concepts like sets, relations, and weighted functions are implicitly used to define the graph structure. Additionally, computational efficiency is analysed using time and space complexity, which is critical for scalability in larger networks. Overall, the project integrates algorithmic design, mathematical reasoning, and structured programming in C++ to solve a real-world navigation and route optimization problem effectively.

2 Literature Review

Route planning systems shows that most solutions are based on graph theory and shortest path algorithms, particularly Dijkstra's Algorithm for networks with non-negative weights. Transportation networks are modelled as weighted graphs, with nodes as locations and edges as roads. Other algorithms like Bellman-Ford and A* are also used for handling negative weights or improving efficiency in large-scale systems.

Existing methodologies focus on efficient data representation using adjacency lists and priority queues to optimize computation. Advanced navigation systems enhance these methods by incorporating dynamic factors such as traffic conditions and using techniques like bidirectional search to improve performance.

This project implements a simplified route planner using C++ and basic data structures, focusing on core concepts for academic understanding. It bridges theory and practice while allowing scope for future enhancements like real-time data and graphical interfaces.

3 Objectives

- To design and implement an efficient route planning system using graph-based data structures and algorithms such as Dijkstra's Algorithm for shortest path computation.
- To apply fundamental concepts of data structures like graphs, priority queues, and adjacency lists in solving real-world problems related to navigation and route optimization.
- To develop a user-friendly system capable of computing and displaying the shortest path and total distance between selected locations,demonstrating practical implementation of algorithmic problem-solving in C++

4 Methodology

4.1 Architectural Diagram

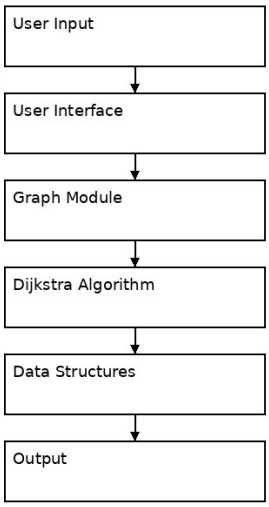

Figure 1: Architectural Diagram

4.2 Hardware and Software Requirements

Hardware Requirements:

The hardware requirements for implementing the Route Planner / Map Navigator project are minimal, as it is a lightweight console-based application:

>Processor: Intel i3 or higher.

> RAM: Minimum 4 GB

> Storage: At least 500 MB free space
> Display: Standard monitor.
> Input Devices: Keyboard and mouse

Software Requirements:

The software requirements include the programming environment and tools necessary for development and execution:

> Operating System: Windows / Linux / macOS.
> Programming Language: C++.
> Compiler: GCC / MinGW
> IDE/Editor: Dev-C++, Code::Blocks, or Visual Studio Code
> Libraries Used: Standard Template Library (STL)
~Vector, queue, iostream.

4.3 Working Principle

The Route Planner project is implemented in **C++** using the **Standard Template Library (STL)**. It uses vector for graph representation (adjacency list), priority_queue for efficient path selection, and arrays/vectors for storing distances and parent nodes. The program is modular, using functions and a Graph class for better organization.

The core functionality is based on Dijkstra's Algorithm, which computes the shortest path efficiently. The system also reconstructs and displays the route along with total distance. A simple console-based interface allows user input, making the program interactive and easy to extend with additional features.

4.4 Software Implementation

The project uses Dijkstra's Algorithm to find the shortest path between two locations in a graph. All node distances are initially set to infinity except the source node. A priority queue is used to repeatedly select the node with the minimum distance and update its neighbouring nodes if a shorter path is found.

After processing, the shortest distance to the destination is obtained, and the path is reconstructed using a parent array. The program then displays the optimal route along with the total distance, efficiently solving the route planning problem.

Program:

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <climits>
using namespace std;

typedef pair<int, int> pii; // (distance, node)

class Graph {
    int V;
    vector<vector<pii>> adj;

public:
    Graph(int v) {
        V = v;
        adj.resize(V);
    }

    void addEdge(int u, int v, int w) {
        // Input validation
        if (u >= V || v >= V || u < 0 || v < 0) {
            cout << "Invalid edge\n";
            return;
        }
        adj[u].push_back({v, w});
        adj[v].push_back({u, w}); // undirected graph
    }

    void dijkstra(int src, int dest) {
        vector<int> dist(V, INT_MAX);
        vector<int> parent(V, -1);

        priority_queue<pii, vector<pii>, greater<pii>> pq;

        dist[src] = 0;
        pq.push({0, src});

        while (!pq.empty()) {
            int d = pq.top().first;
            int u = pq.top().second;
            pq.pop();

            // Skip outdated entries (important optimization)
            if (d > dist[u]) continue;

            for (auto edge : adj[u]) {
                int v = edge.first;
                int weight = edge.second;

                // Prevent overflow + relaxation
                if (dist[u] != INT_MAX && dist[u] + weight < dist[v]) {
                    dist[v] = dist[u] + weight;
                    pq.push({dist[v], v});
                    parent[v] = u;
                }
            }
        }

        // Check if path exists
        if (dist[dest] == INT_MAX) {
            cout << "No path exists\n";
            return;
        }
        cout << "Shortest Distance: " << dist[dest] << endl;

        // Reconstruct path
        vector<int> path;
        for (int v = dest; v != -1; v = parent[v])
            path.push_back(v);

        cout << "Path: ";
        for (int i = (int)path.size() - 1; i >= 0; i--)
            cout << path[i] << " ";
        cout << endl;
    }
};
int main() {
    int V = 6;
    Graph g(V);

    g.addEdge(0, 1, 4);
    g.addEdge(0, 2, 1);
    g.addEdge(2, 1, 2);
    g.addEdge(1, 3, 1);
    g.addEdge(2, 3, 5);
    g.addEdge(3, 4, 3);
    g.addEdge(4, 5, 2);
    int src = 0, dest = 5;
    g.dijkstra(src, dest);
    return 0;
}
```

5 Results and Discussion

1. Summary of Key Results

The implemented Route Planner successfully computes the shortest path between two locations using Dijkstra's Algorithm. For the given graph with 6 nodes, the program found the shortest distance from source (0) to destination (5) as **9**.

The optimal path identified is: **0 → 2 → 1 → 3 → 4 → 5**.

The program efficiently handled the graph using adjacency lists and a priority queue, ensuring accurate results with minimal time complexity.

2. Interpretation

- The results confirm that Dijkstra's Algorithm is effective for finding the shortest path in graphs with non-negative edge weights.
- The computed distance of 9 represents the minimum travel cost between the chosen locations in the network.
- The path **0 → 2 → 1 → 3 → 4 → 5** indicates that the algorithm correctly explored multiple routes and selected the most optimal one.

3. Achievement of Objectives

The main objectives of the study were:

- To design and implement a route planner using graph and Dijkstra's Algorithm.
- To find the shortest distance and path between given locations.
- To display the optimal route to the user.

All these objectives were successfully achieved. The system correctly computes the shortest distance and presents the exact route, fulfilling the intended goals.

4. Implications

- The project demonstrates how classical algorithms can be applied to solve real-world problems like navigation and route planning.
- It can be extended to larger and dynamic networks with real-time data such as traffic conditions, road closures, and time-based weights.
- The system can be integrated into GPS, logistics, and transportation planning applications to provide efficient and cost-effective routing solutions.

6 Mapping POs and SDGs

6.1 Program Outcomes (POs) Mapping

Table 1: Mapping of Program Outcomes (POs) to Project Relevance

PO No.	Program Outcome	Relevance
PO2	Problem Analysis	The project models the route planning problem using graph theory and shortest path algorithms.
PO3	Design/Development of Solutions	Designed a graph-based system to compute shortest paths efficiently.
PO5	Engineering Tool Usage	Implemented using C++ and STL (vector, priority queue).
PO9	Communication	The project enhances communication skills by requiring clear documentation, explanation of algorithms, and presentation of results
PO10	Project Management and Finance	Project involved planning, coding, testing, and documentation.

6.2 Sustainable Development Goals (SDGs) Mapping

Table 2: Mapping of Sustainable Development Goals (SDGs) to Project Relevance

SDG No.	Goal	Relevance
SDG 9	Industry, Innovation and Infrastructure	Supports smart navigation systems
SDG 11	Sustainable Cities and Communities	Improves urban mobility
SDG 12	Responsible Consumption and Production	Reduces fuel usage
SDG 13	Climate Action	Minimizes emissions

7 Conclusion

The Route Planner project successfully demonstrates the implementation of graph-based algorithms to solve real-world navigation problems. Using Dijkstra's Algorithm, the system efficiently computes the shortest path between locations. The project achieves its objectives and can be further enhanced with real-time data and graphical interfaces.

Further improvements include developing a **graphical user interface (GUI)** for better user interaction and using **city names instead of numeric nodes** for usability. Advanced algorithms such as A* can also be implemented to improve performance for large-scale networks. Additionally, the system can be extended to support **multiple routes comparison**, shortest-time paths, and large datasets for real-world applications.

Bibliography

E. W. Dijkstra, *"A note on two problems in connexion with graphs,"* Numerische Mathematik, 1959.

T. H. Cormen , C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, MIT Press.

P. E. Hart, N. J. Nilsson, B. Raphael, *"A Formal Basis for the Heuristic Determination of Minimum Cost Paths,"* IEEE Transactions, 1968.

Thomas H. Cormen , *Data Structures and Algorithms*, MIT OpenCourseWare .

GeeksforGeeks, *Dijkstra's Algorithm* (https://www.geeksforgeeks.org)

TutorialsPoint , *Data Structures in C++*